

FONDAMENTI DI INFORMATICA T-2 – Lab 12

L'associazione delle compagnie aeree ha richiesto un'applicazione per effettuare ricerche e prenotazioni dei voli diretti da un aeroporto a un altro nella data specificata.

DESCRIZIONE DEL DOMINIO DEL PROBLEMA.

Secondo le convenzioni internazionali, ogni città con aeroporto è identificata da un codice univoco di 3 lettere; se tale aeroporto è unico, la sigla vale anche per l'aeroporto (es. "BLQ" = Bologna), mentre nel caso di più aeroporti ognuno ha la propria sigla (es. "LHR" per London Heathrow, "LGW" per London Gatwick, "STN" per London Stansted), diversa da quella della città in quanto tale (nell'esempio, "LON" per London).

Ogni volo è descritto da un identificativo univoco (costituito a sua volta da due parti: la sigla della compagnia [due o tre caratteri] e un numero identificativo [da 2 a 4 cifre]), le sigle degli aeroporti di partenza e arrivo, i rispettivi orari di partenza e arrivo *rispetto all'ora locale*, l'offset del giorno (0 se si arriva lo stesso giorno della partenza, 1 se si arriva il giorno dopo la partenza, -1 se si arriva il giorno prima della partenza), i giorni di effettuazione (per convenzione espressi da una sequenza di numeri dove 1=lunedì,..., 7=domenica, o un trattino se quel giorno il volo non opera), e per finire il tipo di aeromobile (da cui dipende ovviamente il numero di posti disponibili a bordo). Se il volo ha orari diversi in diversi giorni, o è effettuato con aeromobili diversi in giorni o periodi diversi, è descritto da più righe, come negli esempi seguenti.

ESEMPI:

AZ1344,BLQ,14:30,CTA,16:15,0,1234567,A320

volo giornaliero

OK733,BLQ,06:15,PRG,08:10,0,12-456-,AT4

volo non operante mercoledì e domenica

FR9367,BLQ,14:25,GRO,16:15,0,----5--,738

FR9367,BLQ,14:40,GRO,16:30,0,-2-4-6-,738

volo giornaliero operante con orari diversi nei vari giorni

FR9367,BLQ,20:35,GRO,22:25,0,1-----,738

FR9367,BLQ,21:25,GRO,23:15,0,--3---7,738

AZ894,FCO,22:00,CAI,02:20,0,----56-,332

volo giornaliero operato con aerei diversi nei vari giorni

AZ894,FCO,22:00,CAI,02:20,0,1234--7,321

Le descrizioni dei voli previsti in orario sono disponibili nel file di testo [FlightSchedule.txt](#), mentre l'elenco delle sigle degli aeroporti e delle città, *coi relativi fusi orari*, è archiviato nel file [Cities.txt](#); i tipi di aerei, con i posti disponibili in ciascuno, sono disponibili in un terzo file di testo, di nome [Aircrafts.txt](#) (tutti descritti nel seguito).

Parte 1

Dati (*namespace flights.model*)

Il modello dei dati deve essere organizzato secondo il diagramma UML riportato a pagina 2.

SEMANTICA:

- la classe **Aircraft** (fornita nello Start Kit) rappresenta un tipo di aereo, caratterizzato da un identificativo univoco e dal numero di posti offerti; il metodo **getSeats** restituisce il numero di posti disponibili a bordo;
- la classe **Airport** (fornita nello Start Kit) rappresenta un aeroporto: ogni istanza è caratterizzata dalla sua sigla (id univoco) e dalla denominazione estesa corrispondente, e dispone di un metodo **getCity** che restituisce la città in cui l'aeroporto si trova, sotto forma di istanza di **City**;
- la classe **City** (fornita nello Start Kit) rappresenta una città: ogni istanza è caratterizzata dalla sua sigla (id univoco), dalla denominazione estesa corrispondente, dal fuso orario locale e dall'insieme dei suoi aeroporti; il metodo **getAirports** restituisce gli aeroporti della città (sotto forma di **Set<Airport>**), mentre il metodo **getCurrentTimeZone** restituisce il fuso orario corrente^(*) della città, riferito a GMT (un intero fra -12 e +12);
- la classe **FlightSchedule** (da realizzare) rappresenta la descrizione di un volo con tutte le proprietà dettagliate sopra. Il costruttore dovrà verificare i parametri in ingresso generando le opportune eccezioni (NullPointerException in caso di parametri null, IllegalArgumentException nei restanti casi) oltre a verificare la coerenza tra la data di partenza e quella di arrivo. **FlightSchedule** deve inoltre fornire gli opportuni accessor e deve

^(*) L'ora legale si considera inglobata nel valore restituito: pertanto, l'Italia sarà su +1 in inverno e su +2 in estate, e così per ogni altra località.

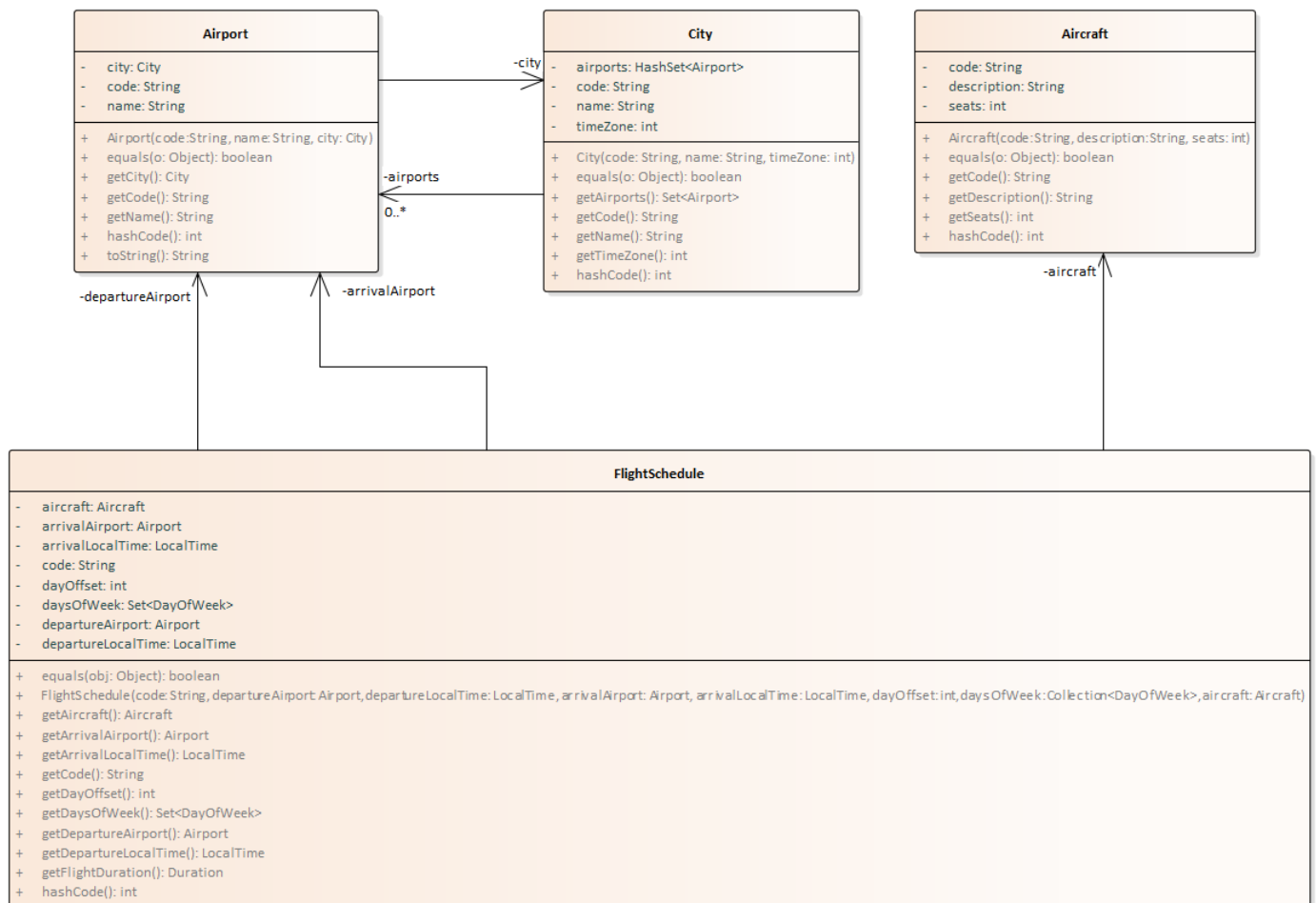
disporre di un metodo `getFlightDuration` che restituisce la durata del volo (`java.time.Duration`), *tenendo opportunamente conto dei fusi orari*^(*);

Al fine di calcolare la durata del volo, occorre prima integrare data e ora di partenza e fuso orario della città di partenza in un `OffsetDateTime`, poi fare lo stesso con data e ora di arrivo e fuso orario della città di arrivo; successivamente si può calcolare la durata del volo (come è noto, è possibile calcolare una `Duration` partendo da due `OffsetDateTime`) e, nel caso in cui questa sia negativa^(*), è sufficiente sommare un giorno. Per semplicità, di seguito si può trovare il codice per la costruzione degli `OffsetDateTime` di cui sopra:

```
OffsetDateTime departure =
    OffsetDateTime.of(LocalDate.now(),
        departureLocalTime,
        ZoneOffset.ofHours(getDepartureAirport().getCity().getTimeZone()));

OffsetDateTime arrival =
    OffsetDateTime.of(LocalDate.now().plusDays(getDayOffset()),
        arrivalLocalTime,
        ZoneOffset.ofHours(getArrivalAirport().getCity().getTimeZone()));
```

Lo StartKit contiene anche i test (da includere nel progetto) per verificare il funzionamento della classe `FlightSchedule`.



^(*) Si noti che un volo verso est può dar luogo a durate apparentemente negative, da correggere aggiungendo 24h.

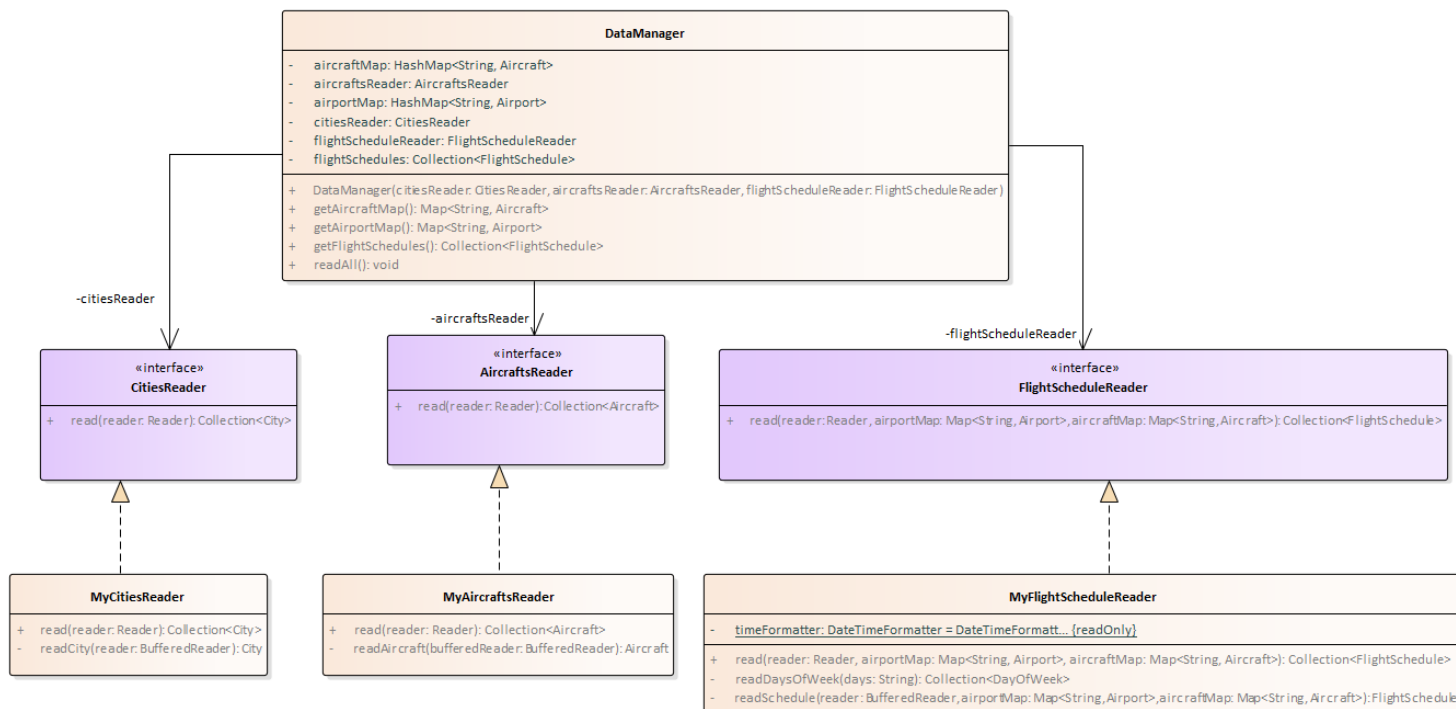
Es.: New York 19.00 / Roma 9.30, durata apparente -9h30, differenza fuso orario +6h, durata reale -15h30, ovvero 8h30 (mod 24h)

Es.: Roma 22.00 / Cairo 2.20, durata apparente -19h40, differenza fuso orario +1h, durata reale -20h40, ovvero 3h20 (mod 24h)

^(*) Si noti che un volo verso est può dar luogo a durate apparentemente negative, da correggere aggiungendo 24h.

Es.: New York 19.00 / Roma 9.30, durata apparente -9h30, differenza fuso orario +6h, durata reale -15h30, ovvero 8h30 (mod 24h)

Es.: Roma 22.00 / Cairo 2.20, durata apparente -19h40, differenza fuso orario +1h, durata reale -20h40, ovvero 3h20 (mod 24h)



I reader di seguito descritti sono tutti dotati di un metodo *read* che restituisce una *collection* specifica di oggetti e che dichiara di rilanciare *IOException* in caso di problemi di lettura, oppure *BadFileFormatException* in caso di problemi nel formato del file.

- Il file di testo *Cities.txt* contiene l'elenco delle sigle delle città aeroportuali con le rispettive informazioni, una per riga. Ogni riga contiene nell'ordine i seguenti campi, separati da virgole:
 - la sigla della città (3 caratteri)
 - il suo fuso orario nel formato **GMT±N** (soltanto **GMT** nel caso **N** sia zero)
 - la denominazione della città
 - nel solo caso di più aeroporti: una lista di aeroporti coi relativi nomi, separati fra loro da virgole, ciascuno nella forma **sigla/nome**

ESEMPI (orario estivo, con ora legale):

LON, GMT+1, London, LHR/Heathrow, LGW/Gatwick, STN/Stansted
BLQ, GMT+2, Bologna

La classe *MyCitiesReader* (fornita nello Start Kit) implementa l'interfaccia *CitiesReader* (pure fornita nello Start Kit): le eccezioni lanciate dal metodo di lettura sono le stesse descritte sopra.

- Il file di testo *Aircrafts.txt* contiene l'elenco delle sigle dei possibili aerei, una per riga. Ogni riga contiene nell'ordine i seguenti campi, separati da virgole:
 - la sigla dell'aereo
 - la sua descrizione estesa
 - il numero di posti offerti

ESEMPI

738, Boeing 737-800, 189
AT4, ATR-42, 46
321, Airbus A-321, 185

La classe *MyAircraftsReader* (fornita nello Start Kit) implementa l'interfaccia *AircraftsReader* (pure fornita nello Start Kit): le eccezioni lanciate dal metodo di lettura sono le stesse descritte sopra.

- Il file di testo *FlightSchedule.txt* contiene l'orario dei voli programmati, uno per riga; ogni riga segue il formato mostrato alla pagina precedente e qui di seguito descritto (i vari campi sono separati da virgole):
 - identificativo del volo (fino a 6 caratteri)

- sigla dell'aeroporto di partenza (3 caratteri) e orario di partenza nel formato **hh.mm** (locale italiano)
- sigla dell'aeroporto di arrivo (3 caratteri) e orario di arrivo nel formato **hh.mm** (locale italiano)
- offset in giorni: **0** se si arriva lo stesso giorno della partenza, **1** se si arriva il giorno dopo la partenza, **-1** se si arriva il giorno prima della partenza
- giorni di effettuazione (7 caratteri, da **1** a **7** nell'ordine, o **"-"** se quel giorno il volo non si effettua)
- tipo di aeromobile (fino a 4 caratteri).

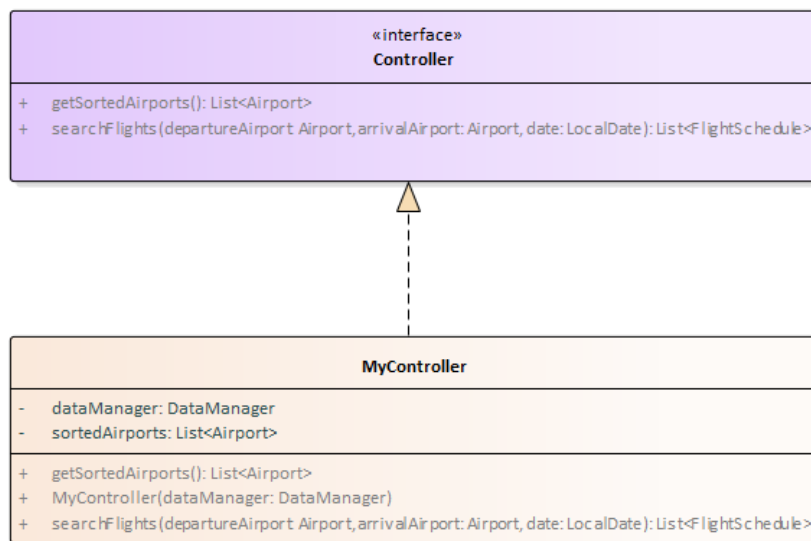
L'interfaccia **FlightScheduleReader** (fornita nello **StartKit**) dichiara i metodi utili per ottenere un insieme di **FlightSchedule** da un generico **Reader**. La classe **MyFlightScheduleReader** (da realizzare) la implementa adottando come formato del testo quello sopra illustrato; in caso di problemi di accesso al file, il metodo di lettura deve lanciare una **IOException**, mentre in caso di errori nel formato del file in lettura deve lanciare l'eccezione **BadFileFormatException** (fornita nello **StartKit**). Si noti che il metodo di lettura prende in ingresso due mappe, una che mette in relazione sigle degli aeroporti e aeroporti corrispondenti ed una che mette in relazione sigle degli aeromobili ed aeromobili corrispondenti; tali mappe devono essere utilizzate durante la lettura per ottenere gli oggetti corrispondenti agli acronimi letti nel file, così da creare correttamente gli oggetti **FlightSchedule**.

La classe **DataManager** (da realizzare), utilizza i vari *reader* (che acquisisce tramite il costruttore sotto forma di riferimenti alle interfacce) e coordina la lettura dei vari file che avviene nel metodo **readAll**. Tale metodo effettua prima la lettura del file degli aeroporti (tramite l'apposito *reader*) e popola la corrispondente mappa, poi effettua la lettura del file degli aeromobili (tramite l'apposito *reader*) e popola la corrispondente mappa, infine effettua la lettura degli orari dei voli (tramite l'apposito *reader*) passando le mappe appena popolate. I metodi *getter* della classe (da realizzare come specificato nel diagramma UML) consentono di recuperare i dati letti.

Lo **StartKit** contiene anche i test relativi alle classi **MyFlightScheduleReader** e **DataManager**.

Parte 2

Controller (namespace *flights.controller*)



La classe **MyController** (da realizzare) implementa l'interfaccia **Controller** (fornita nello **Start Kit**); in particolare:

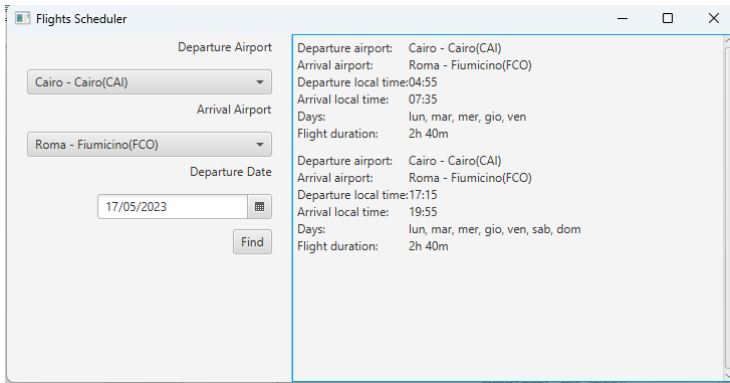
- il metodo **getSortedAirports** restituisce una lista di aeroporti disposti in ordine alfabetico in primis rispetto al nome della città e in subordine al nome dell'aeroporto attraverso la definizione di un opportuno Comparatore;
- il metodo **searchFlights** cerca e restituisce l'elenco dei voli disponibili fra gli aeroporti specificati nella data specificata (a tal fine occorre dedurre, tramite la data, il

giorno della settimana).

Interfaccia utente a console (namespace *flights.ui.console*)

- La classe **FlightConsoleUi** (nello **Start Kit**) realizza l'interfaccia utente a console; il costruttore prende in ingresso un **Controller**, mentre il metodo **run()** consente di visualizzare l'interfaccia. Mediante l'utilizzo della classe **Menu** (fornita nello **Start Kit**), richiedere all'utente la scelta dell'aeroporto di partenza, poi dell'aeroporto di arrivo (recuperare l'elenco degli aeroporti mediante l'utilizzo del metodo **getSortedAirports** di **Controller**); successivamente, richiedere la data del viaggio (convertire la data in un oggetto **Date** mediante l'uso di un opportuno **DateTimeFormatter**). Una volta recuperati tutti i dati, mediante l'utilizzo del metodo **searchFlights** di **Controller**, recuperare l'elenco dei **FlightSchedule** che, successivamente, vanno stampati a console.

Interfaccia utente a finestra (namespace *flights.ui*)



L'interfaccia utente deve essere simile (non necessariamente identica) all'esempio mostrato nella figura che segue. La classe **MainPane** (da realizzare) realizza la finestra principale, che consente di cercare un volo fra due aeroporti per una certa data. A tal fine, due caselle a discesa (**ComboBox**) sono preventivamente popolate mediante il metodo *getSortedAirports* del controller: il testo mostrato in esse dev'essere quello restituito dalla *toString* di **Airport**. Sotto, un componente **DatePicker** consente di

specificare la data del viaggio.

Premendo il **bottone Find** viene invocato il metodo *searchFlights* del controller che, come già detto, restituisce l'insieme dei voli disponibili, con i rispettivi dettagli; tale insieme deve essere quindi inserito in un pannello di tipo **FlightScheduleListPanel** (fornito nello *Start Kit*) che consente di visualizzare tutti i voli selezionati ed è posizionato sulla destra rispetto ai controlli di selezione.

Classi da realizzare per ogni parte

Parte 1: **FlightSchedule**

Parte 2: **MyFlightScheduleReader**, **DataManager**

Parte 3: **MyController**, **MainPane**